

Architecture and Implementation of Pure-Function Audio and Speech Processing Atoms

The development of high-performance, deterministic pipelines for audio and speech processing necessitates the decomposition of complex feature extraction and alignment operations into discrete, mathematically rigorous primitives. These primitives—or atoms—must operate as pure functions, strictly mapping input tensors and configurations to output tensors without side effects, global state dependencies, or file input/output operations. By adhering to functional programming paradigms within the NumPy ecosystem, numerical stability, array shape determinism, and computational efficiency are guaranteed across varying execution environments.

This analysis exhaustively details best-in-class implementations for audio preprocessing, spectral feature extraction, data augmentation, source separation, and temporal alignment algorithms. The objective is to synthesize theoretical foundations with explicit technical contracts for integration into the sciona-atoms-signal repository, addressing critical considerations spanning algorithmic complexity, numerical stability, and natural input contracts.

Foundational Audio Temporal Manipulation

The modification of audio signals in the time domain represents the foundational layer of any speech processing pipeline. Operations at this stage must be meticulously designed to avoid aliasing artifacts, manage spectral leakage, and minimize memory allocation overhead, establishing a pristine signal for downstream spectral transformation.

Band-Limited Polyphase Resampling

Resampling an audio signal from an original sampling rate to a target sampling rate is rarely a trivial operation of decimation or linear interpolation. A naive downsampling approach without appropriate filtering introduces severe spectral leakage and aliasing, permanently destroying high-frequency data and corrupting the target band. While Fourier-domain resampling algorithms—such as `scipy.signal.resample`—exist, they inherently assume that the input signal is periodic. Consequently, when processing non-periodic audio signals, the Fourier method often yields significant boundary artifacts and proves computationally inefficient if the signal length contains large prime factors.³

The optimal implementation for general-purpose audio utilizes polyphase filtering.⁴ This technique conceptually upsamples the signal by an integer factor P , applies a zero-phase low-pass finite impulse response (FIR) filter to strictly restrict the bandwidth to the Nyquist

frequency of the lower sample rate, and subsequently downsamples by an integer factor Q .⁴ However, computing the intermediate upsampled signal is computationally wasteful. The polyphase approach mathematically reorganizes the FIR filter into a matrix of subfilters based on the Noble Identities.² These subfilters are applied through a commutator switch, meaning the algorithm only calculates the exact samples that will survive the final decimation step.²

This method preserves the integrity of the acoustic information while providing substantial computational advantages over brute-force band-limited sinc interpolation in the time domain, provided the ratio of the target to original sample rate can be expressed as a rational fraction.³

Name: `resample_audio`

Description: Resample a discrete-time audio signal using polyphase filtering and a zero-phase low-pass FIR anti-aliasing filter to prevent spectral aliasing.

Source: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample_poly.html

License: BSD-3-Clause

Concept type: `signal_transform`

Signature: `(signal: NDArray, orig_sr: int, target_sr: int) -> NDArray`

Pure function boundary: 1D or 2D waveform arrays and integer sample rates in; 1D or 2D resampled arrays out. No audio file I/O or stateful filter objects.

Contracts:

- require: `orig_sr > 0` and `target_sr > 0`
- require: `signal.ndim` in (1, 2)
- ensure: `result.shape[-1] == ceil(signal.shape[-1] * target_sr / orig_sr)`
Witness: a 100-sample array of ones resampled from 10Hz to 20Hz returns a 200-sample array of ones; frequency components above the new Nyquist limit are suppressed.
Dependencies: `scipy` (`signal.resample_poly`)
CDG stages covered: `bengali_speech/audio_resample`, `birdclef/audio_resample`

Memory-Efficient Audio Windowing and Chunking

Spectral analysis and local frame-level predictions fundamentally require framing a continuous one-dimensional audio signal into overlapping short-time chunks. Instead of physically copying data in memory—which rapidly drains system resources during large batch operations—NumPy's internal memory strides can be manipulated to create a highly efficient sliding window view of the original array.⁷

The `numpy.lib.stride_tricks.sliding_window_view` function (often implemented fundamentally via `as_strided`) allows a 1D array of shape $(N,)$ to be mathematically viewed as a 2D array of

shape *(num_frames, window_size)* simply by configuring the byte offsets (strides) mapped to the system memory.⁷ Because this operation returns a view pointing to the same underlying memory blocks, modifying any element in the resulting view modifies the original array and inadvertently alters multiple overlapping frames. Therefore, the output must be strictly locked as read-only within a pure-function context to prevent side effects.⁷

Name: audio_windows

Description: Extract overlapping fixed-length chunks from a continuous audio signal using memory-efficient strided views without data duplication.

Source:

https://numpy.org/devdocs/reference/generated/numpy.lib.stride_tricks.sliding_window_view.html

License: BSD-3-Clause

Concept type: signal_transform

Signature: (signal: NDArray, window_size: int, hop_size: int) -> NDArray

Pure function boundary: 1D continuous array and integer window parameters in; 2D strided view out. No data copying or state mutation.

Contracts:

- require: signal.ndim == 1
 - require: window_size > 0 and hop_size > 0
 - require: signal.shape[0] >= window_size
 - ensure: result.shape == ((signal.shape[0] - window_size) // hop_size + 1, window_size)
 - ensure: result.flags.writeable == False
- Witness: an array of integers 0 to 9 chunked with size 3 and hop 2 yields exactly `[ , , ]`.
- Dependencies: numpy
- CDG stages covered: cornell_birdcall/chunking, birdclef/chunking

Spectral Feature Extraction Operations

Transforming raw audio into a feature representation suitable for deep neural networks typically involves mapping the time-domain signal into a perceptually relevant time-frequency domain, most prominently the log-Mel spectrogram. This monolithic process can and should be mathematically decoupled into composable, functional atoms.

Short-Time Fourier Transform (STFT) Magnitude Computation

The discrete time-domain signal is systematically divided into overlapping frames using a windowing function. A Hann window is canonically applied to each frame to taper the edges to zero, significantly reducing the spectral leakage that arises from the finite observation

window.⁹ Following the application of the window, the discrete Fourier transform (computed via the highly optimized Fast Fourier Transform or FFT) is applied.

The absolute value of the complex FFT output yields the magnitude spectrum, and squaring it yields the power spectrum, eliminating the phase components which are generally considered less critical for pattern recognition in speech and acoustic event detection.¹¹

Name: `stft_magnitude`

Description: Compute the magnitude spectrogram of a continuous audio signal using the Short-Time Fourier Transform, applying a Hann window to mitigate spectral leakage.

Source: <https://numpy.org/doc/stable/reference/routines.fft.html>

License: BSD-3-Clause

Concept type: `signal_transform`

Signature: (signal: NDArray, n_fft: int, hop_length: int, window: str) -> NDArray

Pure function boundary: 1D waveform array and integer constraints in; 2D magnitude spectrum matrix out. No stateful caching of the window array across distinct calls.

Contracts:

- require: $n_fft > 0$ and $(n_fft \& (n_fft - 1) == 0)$
- require: $hop_length > 0$
- ensure: $result.shape == 1 + n_fft // 2$

Witness: passing a zero-filled array yields a strictly zero-filled matrix of shape $(n_fft // 2 + 1, frames)$.

Dependencies: `numpy (np.fft.rfft, lib.stride_tricks)`

CDG stages covered: `standard_audio_feature/stft`

The Mel Filterbank Matrix Construction

The Mel scale maps pure physical frequencies (in Hertz) to human perceptual features, reflecting the physiological reality that human pitch perception does not scale linearly.¹³ Differences in low frequencies are easily perceived, while equal absolute differences in high frequencies are indistinguishable. To replicate this mathematically, the linear frequency bins output by the STFT are grouped and weighted using a filterbank matrix.

Two primary mathematical formulations exist for this conversion: the HTK formula and the Slaney (Auditory Toolbox) formula.¹⁴ The HTK formula relies on a purely logarithmic mapping defined precisely as $m = 2595 \cdot \log_{10}(1 + f/700)$.¹⁶ Conversely, the Slaney formulation is piecewise in nature. It mandates a strictly linear scale below 1000 Hz, employing exactly 15 bands spaced at approximately 66.66 Hz intervals, beginning at 133.33 Hz.¹⁸ Above the 1000 Hz threshold, the sequence transitions into an exponential scale, preserving the derivative step

size precisely at the boundary to avoid jarring discontinuities.¹⁸

Modern speech and acoustic systems predominantly utilize the Slaney formulation combined with area normalization. The Slaney Mel filterbank applies triangular weighting functions to the linear frequency spectrum, maintaining linear spacing below 1 kHz and logarithmic expansion above 1 kHz. Area normalization requires dividing the triangular Mel weights by the absolute width of the specific Mel band in Hz. This ensures that the energy is scaled correctly across varying filter bandwidths, resulting in decreasing peak amplitudes at higher frequencies—a direct mathematical result of the area normalization procedure.¹⁹

Name: `mel_filterbank`

Description: Construct a linear transformation matrix to project linear FFT frequency bins onto the perceptual Mel scale, defaulting to the Slaney piecewise formulation with strict area normalization.

Source: <https://librosa.org/doc/main/generated/librosa.filters.mel.html>

License: ISC

Concept type: `signal_filter`

Signature: `(n_mels: int, n_fft: int, sr: int, fmin: float, fmax: float) -> NDArray`

Pure function boundary: explicit integer and float structural configurations in; 2D static weight projection matrix out.

Contracts:

- require: `sr > 0` and `n_mels > 0` and `n_fft > 0`
- require: `0 <= fmin < fmax <= sr / 2`
- ensure: `result.shape == (n_mels, 1 + n_fft // 2)`
- ensure: `np.all(result >= 0)`

Witness: fixed inputs generate a static sparse matrix where each row sum strictly corresponds to its normalized band area.

Dependencies: `numpy`

CDG stages covered: `dcase_sed/mel_filter`, `birdclef/mel_filter`

Spectrogram Projection and Stable Logarithmic Scaling

Applying the filterbank to the audio frame data is implemented as a highly optimized dense dot product between the transposed Mel basis matrix and the continuous power spectrum array.²⁰

Following the application of the Mel scale, the signal amplitude must be converted to decibels (dB). Because human perception of acoustic intensity is fundamentally logarithmic, neural networks trained on acoustic features converge significantly faster when processing log-scaled inputs.¹²

However, computing the logarithm of a spectrum presents profound numerical stability

challenges. The logarithm of absolute zero is mathematically undefined, yielding a negative infinity (-inf) floating-point representation that instantly destroys gradients during backpropagation.¹⁰ To prevent this numerical underflow, the signal must be clipped using a strict amin threshold. Standard robust implementations compute

$10 \cdot \log_{10}(\max(S_{mel}, amin))$ and subsequently normalize this relative to the maximum power present in the spectrum to establish a defined dynamic range dictated by top_db.¹⁰

Name: apply_mel_filterbank

Description: Project a high-dimensional linear power spectrum onto the compressed perceptual Mel scale via a dot product with a precomputed Mel filterbank matrix.

Source: <https://librosa.org/doc/main/generated/librosa.feature.melspectrogram.html>

License: ISC

Concept type: signal_transform

Signature: (power_spectrum: NDArray, mel_fb: NDArray) -> NDArray

Pure function boundary: 2D power spectrum and 2D filter matrix in; compressed 2D mel spectrum out.

Contracts:

- require: power_spectrum.shape == mel_fb.shape
 - ensure: result.shape == (mel_fb.shape, power_spectrum.shape)
- Witness: passing conformant non-negative matrices yields a strictly non-negative dot product matrix.
- Dependencies: numpy (np.dot)
- CDG stages covered: birdclef/mel_spectrogram

Name: log_mel_spectrogram

Description: Convert a linear mel spectrogram to a decibel (logarithmic) scale, employing strict numerical stability flooring to eliminate gradient-destroying infinite values.

Source: https://librosa.org/doc/0.9.2/generated/librosa.power_to_db.html

License: ISC

Concept type: signal_transform

Signature: (mel_spectrum: NDArray, ref: float, amin: float, top_db: float) -> NDArray

Pure function boundary: 2D data array and distinct float boundaries in; 2D log-scaled array out.

Contracts:

- require: amin > 0 and top_db > 0
- require: ref > 0

- ensure: `np.all(np.isfinite(result))`
- ensure: `np.min(result) >= -top_db`
 Witness: a spectrum containing absolute zero values correctly floors to precisely `-top_db` instead of yielding `-inf`.
 Dependencies: `numpy`
 CDG stages covered: `birdclef/log_mel`, `standard_audio_feature/log_mel`

Mel-Frequency Cepstral Coefficients (MFCC) Extraction

For applications requiring highly decorrelated, pitch-invariant representations—such as classic speaker identification algorithms or distinct bird call detection pipelines—the log-mel spectrogram is subjected to a final mathematical transformation.²³ The Discrete Cosine Transform (DCT), specifically DCT Type-II, is applied across the Mel frequency bins.

This transformation treats the spectrum itself as a signal and extracts its frequencies (the cepstrum). By retaining only the first N coefficients of the DCT output, the algorithm successfully isolates the broad spectral envelope of the sound—which corresponds to the shape of the vocal tract—while systematically discarding the fine harmonic structure indicative of raw pitch.²⁴

Name: `mfcc`

Description: Extract distinct Mel-Frequency Cepstral Coefficients by applying a Type-II Discrete Cosine Transform to a log-mel spectrogram and truncating the output.

Source: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.fft.dct.html>

License: BSD-3-Clause

Concept type: `signal_transform`

Signature: `(log_mel_spectrum: NDArray, n_mfcc: int) -> NDArray`

Pure function boundary: 2D log-mel array in; 2D truncated DCT coefficient array out.

Contracts:

- require: `0 < n_mfcc <= log_mel_spectrum.shape`
- ensure: `result.shape == (n_mfcc, log_mel_spectrum.shape)`
 Witness: an input matrix mapped through the DCT operation correctly and deterministically subsets to exactly `n_mfcc` rows.
 Dependencies: `scipy (fft.dct)`
 CDG stages covered: `birdclef/mfcc`

Signal Augmentation and Output Smoothing Strategies

Building robust deep learning models for acoustic environments requires aggressive perturbation of the input data to prevent overfitting, alongside the mathematical smoothing of output frame predictions to yield coherent physical events.

Spectrogram Augmentation (SpecAugment)

Introduced by Park et al. in 2019, SpecAugment is a highly efficient augmentation policy that operates directly on the computed log-mel spectrogram rather than incurring the computational expense of manipulating the raw audio waveform.²⁶ The full policy incorporates time warping alongside time and frequency masking, though time warping is frequently omitted due to its high computational cost relative to its marginal performance impact.²⁸

The masking operations simulate temporary microphone failures, acoustic occlusion, or unpredictable noise transients. Time masking randomly selects a uniform starting point t_0 and a width t , subsequently replacing all temporal consecutive time steps $[t_0, t_0 + t)$ with a defined imputation value (usually absolute zero or the dataset mean).²⁹ Frequency masking mirrors this identical logic across the vertical frequency axis, zeroing out random consecutive Mel frequency channels.³¹ In a pure functional NumPy paradigm, SpecAugment relies on replacing these contiguous bands of time and frequency data with a constant value, simulating data dropout. The grid-like removal of data appears as stark, uniform dark bands representing the minimum value, showing both a horizontal frequency mask and a vertical time mask. This explicit zeroing out acts as a structural dropout mechanism on the time-frequency representation, forcing the neural network to learn robust features even when critical parts of the speech signal are completely missing.³⁰

Name: spec_augment_time_mask

Description: Apply structured time masking to a spectrogram by deterministically zeroing out random contiguous vertical bands to simulate temporal data loss.

Source: <https://arxiv.org/abs/1904.08779>

License: Public Domain (Algorithmic Concept)

Concept type: signal_filter

Signature: (spectrogram: NDArray, num_masks: int, max_width: int, mask_value: float, rng_seed: int) -> NDArray

Pure function boundary: 2D data array and an explicit integer seed in; 2D masked array out.
Uses deterministic seed for isolated PRNG state.

Contracts:

- require: num_masks >= 0 and max_width > 0
 - ensure: result.shape == spectrogram.shape
- Witness: passing an identical array with the identical known seed produces the exact

reproducible vertical zero-bands.

Dependencies: numpy

CDG stages covered: standard_audio_feature/spec_augment

Name: spec_augment_freq_mask

Description: Apply structured frequency masking to a spectrogram by deterministically zeroing out random contiguous horizontal bands to simulate missing spectral frequencies.

Source: <https://arxiv.org/abs/1904.08779>

License: Public Domain (Algorithmic Concept)

Concept type: signal_filter

Signature: (spectrogram: NDArray, num_masks: int, max_width: int, mask_value: float, rng_seed: int) -> NDArray

Pure function boundary: 2D data array and an explicit integer seed in; 2D masked array out. Uses deterministic seed for isolated PRNG state.

Contracts:

- require: num_masks >= 0 and max_width > 0
- ensure: result.shape == spectrogram.shape

Witness: passing an identical array with the identical known seed produces the exact reproducible horizontal zero-bands.

Dependencies: numpy

CDG stages covered: standard_audio_feature/spec_augment

Post-Processing Frame Smoothing via Median Filtering

Acoustic models predicting at the strict frame level—such as those executing Sound Event Detection (SED) tasks—frequently produce inherently noisy output probabilities. A model might confidently predict the presence of an event for five frames, drop to near-zero confidence for a single frame, and return to high confidence. Temporal smoothing is required to clean these artifacts.

Using a median filter sliding over the 1D sequence of probabilities suppresses this isolated salt-and-pepper noise far more effectively than a standard moving average.³² Given a specific

odd-numbered window size W , the median filter outputs the median value within the local neighborhood. Because it relies on order statistics rather than arithmetic means, it preserves sharp edges and exact acoustic event onset and offset boundaries without blurring them.³³ A pristine pure implementation wraps the standard array logic over NumPy memory strides or utilizes `scipy.signal.medfilt`.³⁴

Name: median_filter_1d

Description: Smooth raw frame-level prediction probabilities using a sliding median order-statistic filter to suppress transient salt-and-pepper noise while preserving event edges.

Source: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.medfilt.html>

License: BSD-3-Clause

Concept type: signal_filter

Signature: (predictions: NDArray, kernel_size: int) -> NDArray

Pure function boundary: 1D probability array in; 1D median-smoothed array out.

Contracts:

- require: kernel_size > 0 and kernel_size % 2 == 1
- require: predictions.ndim == 1
- ensure: result.shape == predictions.shape

Witness: a probability array filtered with a kernel size of 3 strictly corrects to .

Dependencies: scipy (signal.medfilt)

CDG stages covered: dcase_sed/median_smoothing

Advanced Loudness Normalization and Source Separation

Ensuring mathematical consistency across disparate audio files recorded in uncontrolled environments requires rigorous objective normalization. Concurrently, isolating target vocals from dense musical mixtures demands highly specialized masking techniques operating in the complex spectral domain.

EBU R128 Integrated Volume Normalization

Classic audio volume normalization relies on simply detecting the peak signal measurement and scaling the file accordingly. This approach is fundamentally flawed, as highly compressed audio can pass a maximum peak threshold while carrying substantially more acoustic energy, rendering it perceptually deafening.³⁵ To combat this "Loudness War," the European Broadcasting Union formalized the EBU R128 standard, which standardizes perceived volume using the ITU-R BS.1770-4 mathematical algorithm.³⁶

The ITU-R BS.1770-4 algorithm evaluates audio through a series of discrete filters and gates. First, the signal is processed by a specialized K-weighting curve. This curve is mathematically constructed by cascading a high-shelf biquad filter (simulating the acoustic effects of the human head) and a high-pass biquad filter (removing sub-bass frequencies humans poorly perceive).³⁹ The exact discrete biquad coefficients necessary for processing audio at the standard 48 kHz sampling rate are strictly defined in the specification.

Filter Stage	Equation Type	b0	b1	b2	a1	a2
Stage 1: High-Shelf	Biquad Cascade	1.5351248	-2.6916961	1.1983928	-1.6906592	0.7324807
Stage 2: High-Pass	Biquad Cascade	1.0	-2.0	1.0	-1.9900474	0.9900722

Following the application of the K-weighting filter, the signal is segmented into 400ms blocks with a 75% overlap.⁴¹ The energy of each block is calculated as $z_{i,j}$. The absolute loudness for block j is calculated on a logarithmic scale: $L_j = -0.691 + 10 \cdot \log_{10}(\sum G_i \cdot z_{i,j})$.⁴³ The algorithm then applies a strict two-stage gating system. Blocks evaluating below an absolute threshold of -70 LUFS are completely discarded as silence. From the remaining active blocks, an average is calculated, and a secondary relative gate discards any block falling -10 LU below this active average. The final integrated loudness is the logarithmic sum of the blocks surviving both gates.³⁶ Calculating the scalar difference between this measured LUFS and the target LUFS (typically -23 LUFS) yields the exact linear gain multiplier required to normalize the waveform perfectly.³⁵

Name: ebu_r128_normalize

Description: Normalize audio volume to a perceptually consistent target LUFS employing the ITU-R BS.1770-4 K-weighting biquad cascade and rigorous two-stage gated integration.

Source: <https://github.com/csteinmetz1/pyloudnorm/blob/master/pyloudnorm/meter.py>

License: MIT

Concept type: signal_transform

Signature: (signal: NDArray, sr: int, target_lufs: float) -> NDArray

Pure function boundary: 1D or 2D array and a target LUFS float parameter in; scaled array out.

Contracts:

- require: sr > 0
- require: target_lufs <= 0.0
- ensure: result.shape == signal.shape

Witness: supplying a very quiet synthetic sine wave guarantees a mathematically correct proportional gain multiplier is applied across all samples.

Dependencies: numpy, scipy (signal.lfilter)

CDG stages covered: bengali_speech/volume_norm

Complex Source Separation via Wiener Soft Masking

For complex tasks such as vocal isolation—foundational to concepts like HTDemucs and automated lyric alignment—neural networks generally output estimates of the magnitude spectrograms of the distinct sources (e.g., vocal vs. accompaniment). These magnitude estimates must be statistically combined into a mask to be applied to the original complex mixture spectrogram to retrieve the phase information necessary for audio reconstruction.

The fundamental approach generates an Ideal Ratio Mask (IRM). The soft mask matrix for a

specific target source is calculated pointwise as $M = \frac{S_{target}}{S_{target} + S_{noise} + \epsilon}$.⁴⁴ However, simple ratio masking assumes local independence between time-frequency bins. Applying a generalized multichannel Wiener filter over the spectrogram estimates enforces vital cross-channel consistency and refines the mask using iterative expectation-maximization statistics.⁴⁵ The pure functional atom implements this process by determining the Wiener filter coefficients, calculating the stabilized soft mask, and performing an element-wise multiplication against the original complex mixture spectrogram before passing the result to an inverse STFT.⁴⁷

Name: wiener_soft_mask

Description: Compute an Ideal Ratio Mask (IRM) refined by generalized Wiener filtering from estimated magnitude spectrograms to separate a specific sound source while preserving phase.

Source: <https://github.com/sigsep/norbert>

License: MIT

Concept type: signal_transform

Signature: (target_mag: NDArray, noise_mag: NDArray, mix_complex: NDArray, eps: float) -> NDArray

Pure function boundary: parallel real spectrogram matrices and the original complex mixture matrix in; complex isolated target spectrogram matrix out.

Contracts:

- require: target_mag.shape == noise_mag.shape == mix_complex.shape
- require: eps > 0
- ensure: np.all(np.isfinite(result))

Witness: calculating a mask using perfectly identical target and noise matrices

guarantees an exact 0.5 magnitude multiplication across the resulting mask.

Dependencies: numpy

CDG stages covered: alice_lyric/vocal_isolation

Lexical Processing and Temporal Alignment Logic

Mapping literal transcripts to raw recorded audio is a multi-stage process requiring the resolution of orthographic text into standardized phonemes, and subsequently determining the exact temporal boundaries of those individual phonemes against the acoustic frame predictions generated by an acoustic model.

Deterministic Grapheme-to-Phoneme (G2P) Conversion

A pure-function, rule-based G2P conversion engine accurately translates variable strings of graphemes into standardized phonetic sequences without requiring massive, memory-heavy neural networks or maintaining global dictionary states.⁴⁹ Implementations like G_i2P_i (or tools derived from the foundational Polish transcription rules of Maria Steffen-Batóg) utilize directed acyclic graphs and regular expressions to evaluate input characters within specific phonological contexts (e.g., matching the left and right adjacent character structures) against predefined, rigid rule lists.⁴⁹

Critically for forced alignment architectures, the conversion engine must operate as an index-preserving transformation. It is utterly insufficient to merely return a string of phonemes; the atom must return data structures that preserve the exact starting and ending indices mapping the generated phonetic token back to its source orthographic grapheme.⁵⁰ This is the only mechanism that allows a temporal alignment found in the acoustic domain to map back to the original text document.

Name: rule_based_g2p

Description: Convert an orthographic string of graphemes to a sequence of phoneme IDs utilizing strict context-matching rule evaluation, explicitly preserving character-to-phoneme alignment indices.

Source: <https://github.com/roedoejet/g2p>

License: MIT

Concept type: signal_transform

Signature: (text: str, ruleset: dict[str, tuple[str, str, str]]) -> list[tuple[int, int, int]]

Pure function boundary: text string and explicit pre-loaded rule dictionary in; list of (phoneme_id, start_char_idx, end_char_idx) tuples out.

Contracts:

- require: len(text) > 0

- ensure: `len(result) > 0`
- ensure: `start_char_idx <= end_char_idx` for all returned tuples
 Witness: applying a simple substitution rule mapping "ph" to "f" yields the phonetic token while maintaining the correct underlying two-character original text index boundaries.
 Dependencies: python-builtins (re)
 CDG stages covered: `alice_lyric/g2p`

Forced Alignment via Constrained Dynamic Time Warping

Dynamic Time Warping (DTW) calculates the optimal monotonic path through a cost matrix to align an audio sequence with a text sequence. The cost matrix grid represents the distances between every frame of audio and every phoneme, and the winning path traces the lowest cumulative cost from the bottom-left to the top-right.⁵² By visualizing the cost matrix and drawing the optimal path through it, the concepts of monotonicity (no going backwards) and temporal warping (one phoneme spanning multiple audio frames) become mathematically clear.

In forced alignment tasks (e.g., locking predicted phoneme boundaries to a specific audio recording), the standard unconstrained DTW $O(N \cdot M)$ computational complexity is severely restricted to prevent illogical mappings. A strict monotonicity constraint is mathematically enforced to prohibit backtracking, ensuring time only flows forward.⁵² The algorithm generates an accumulated cost matrix by evaluating the fundamental dynamic programming recurrence relation:

$$D(i, j) = d(x_i, y_j) + \min(D(i - 1, j), D(i - 1, j - 1), D(i, j - 1))$$

For forced alignment, valid traversal paths are heavily bounded using specific global windowing algorithms. The Sakoe-Chiba band limits the allowable distance the path can wander from the pure diagonal, significantly narrowing the search space and preventing pathological warpings.⁵⁴ The final backtracking phase of the algorithm evaluates the matrix to recover the specific index tuples $(frame_idx, phoneme_idx)$, finalizing the temporal alignment output.⁵⁶

Name: `dtw_alignment`

Description: Compute the optimal continuous monotonic alignment between frame-level probabilities and a fixed phoneme sequence utilizing Dynamic Time Warping bounded by global Sakoe-Chiba constraints.

Source: <https://dynamictimewarping.github.io/py-api/>

License: MIT (Custom pure NumPy DP matrix implementation required to supersede GPL derivatives)

Concept type: `signal_transform`

Signature: (frame_probs: NDArray, phoneme_sequence: list[int]) -> list[tuple[int, int]]

Pure function boundary: 2D probability matrix and 1D target label sequence in; list of sequence path index tuples out.

Contracts:

- require: frame_probs.shape[0] >= len(phoneme_sequence)
 - ensure: result[0] == (0, 0) and result[-1] == (frame_probs.shape[0]-1, len(phoneme_sequence)-1)
 - ensure: indices are strictly monotonic increasing (no backtracking)
- Witness: executing against a perfectly diagonal probability matrix aligning with a matching sequence returns a strictly diagonal tuple path without deviation.
- Dependencies: numpy
- CDG stages covered: alice_lyric/dtw_alignment

Monotonic Alignment Search (MAS) for Generative Sequences

In modern generative Text-to-Speech frameworks (e.g., Glow-TTS and VITS), strict alignment paths are discovered dynamically during the training loop without relying on external forced aligners. The Monotonic Alignment Search (MAS) leverages specialized dynamic programming to rapidly discover an alignment path that explicitly maximizes the log-likelihood of the data parameterized by a normalizing flow.⁵⁷

The MAS recursion loop efficiently caches and evaluates path probabilities. The core temporal loop advances time step t_y , comparing and merging probabilities v_0 (probability representing a transition from the previous frame to a different token) and v_1 (probability representing remaining on the same token from the previous frame). The algorithm iterates through these calculations, padding invalid historical trajectories with $-\infty$ to forcefully guarantee strict monotonicity.⁶⁰ The result of the trace is a mathematically exact binary hard-alignment mask.⁶¹ Implementing this recursively complex logic in pure NumPy eliminates cumbersome framework dependencies and bypasses significant CUDA synchronization and memory-copy overheads when processing on edge devices.⁶²

Name: monotonic_alignment_search

Description: Compute a strict binary hard-alignment mask that maximizes the transition likelihood between text and speech latent representations utilizing efficient dynamic programming.

Source: <https://github.com/coqui-ai/TTS/discussions/2025>

License: MIT

Concept type: signal_transform

Signature: (value: NDArray, mask: NDArray, max_neg_val: float) -> NDArray

Pure function boundary: 3D probability array and valid boundary mask array in; 3D binary alignment path tensor out.

Contracts:

- require: value.ndim == 3 and mask.ndim == 3
 - ensure: result.shape == value.shape
 - ensure: np.all((result == 0) | (result == 1))
- Witness: passing an identical uniform probability matrix yields a strictly diagonal, monotonically stepped binary trace.
- Dependencies: numpy
- CDG stages covered: alice_lyric/vits_alignment

Connectionist Temporal Classification (CTC) Decoding Mechanisms

Acoustic models trained employing the CTC loss function output dense $N \times V$ probability matrices, where N is the frame length and V is the vocabulary size. Decoding this matrix requires collapsing profound structural redundancies. CTC accommodates this by introducing a specialized blank token, handling scenarios where prolonged phonemes stretch across consecutive acoustic frames without constituting a true repetition in the final transcript.⁶³

Naive Greedy Decoding

The structurally simplest decoding mechanism evaluates the matrix by executing an **argmax** operation across the vocabulary axis for each individual time frame. The resulting raw sequence is subsequently collapsed by eliminating identical consecutive tokens, followed by a final pass to strip all residual blank tokens. While highly efficient and completely deterministic, this method suffers a fatal mathematical flaw: it completely fails to account for alternative, marginally lower-probability paths that, when evaluated collectively, may jointly sum to a significantly higher total sequence likelihood.⁶⁴

Name: ctc_greedy_decode

Description: Extract the singular highest-probability path from a CTC probability matrix, collapsing consecutive token repeats and purging blank delimiter tokens.

Source: Custom / Standard Algorithm Logic

License: MIT

Concept type: signal_transform

Signature: (log_probs: NDArray, blank_id: int) -> list[int]

Pure function boundary: 2D probability array and explicit integer blank ID in; 1D list of evaluated integer token IDs out.

Contracts:

- require: log_probs.ndim == 2
 - require: 0 <= blank_id < log_probs.shape
 - ensure: result does not contain blank_id
- Witness: a probability matrix heavily favoring the token sequence [1, 1, blank, 2, 2] strictly and deterministically outputs.
- Dependencies: numpy
- CDG stages covered: bengali_speech/ctc_decode

Prefix Beam Search with Integrated Language Model Fusion

To decode a CTC matrix optimally, the discrete probabilities of all possible paths yielding the exact same output sequence must be mathematically summed. CTC Prefix Beam Search (as formulated by Hannun et al., 2014) addresses this by iterating sequentially through the time frames while maintaining a truncated, fixed-size dictionary (the beam) of actively evaluated string prefixes.⁶⁵ CTC Prefix Beam Search operates by expanding active string prefixes frame by frame. Because CTC allows blank tokens and repeated characters, a single prefix (like 'A') can be reached via multiple distinct paths (e.g., 'A_', '_A', 'AA'). The branching tree structure traces the branching and merging of logic states. If the new character matches the last character of the prefix, the path branches. The algorithm tracks the probability of ending in a blank and the probability of ending in a non-blank token separately, merging them via logsumexp before pruning the beam.⁶⁶

For each successive time step, and for each valid character in the target vocabulary, the algorithm attempts to extend the active prefixes. Critically, to rigorously handle CTC's repeat collapse rules, the algorithm tracks two distinct probabilities per prefix in logarithmic space:

P_{blank} (the cumulative probability that the active prefix was reached via a path ending in a blank token) and $P_{\text{no_blank}}$.⁶⁵ If a token is appended that identically matches the final token of the current active prefix, the algorithm branches mathematically into a merged case (where the probability is drawn exclusively from P_{blank}) and an unmerged case.⁶⁷

Numerical stability throughout the recursion is maintained exclusively through strict logsumexp operations (the mathematically stable computation of the logarithm of the sum of exponentials). Executing these calculations via direct probability multiplication rapidly causes catastrophic floating-point underflow.⁶⁸ Furthermore, a Language Model (LM)

probability—represented mathematically as $P_{LM}(\text{prefix})^\alpha$ —and a structural insertion

bonus defined as $\beta \cdot |prefix|$ are directly added to the log-space score during each expansion phase. This crucial integration allows linguistic structural priors to seamlessly override ambiguous acoustic uncertainties at the exact moment of evaluation.⁶⁵

Name: ctc_beam_decode

Description: Decode complex CTC probability matrices utilizing rigorous prefix beam search, strictly maintaining log-space stability and recursively tracking blank and non-blank transitional paths independently.

Source: <https://gist.github.com/awni/56369a90d03953e370f3964c826ed4b0>

License: MIT

Concept type: signal_transform

Signature: (log_probs: NDArray, beam_width: int, blank_id: int, alpha: float, beta: float) -> list[tuple[list[int], float]]

Pure function boundary: 2D probability array and specific numeric tuning hyperparameters in; sorted list of (token_list, log_score) evaluation tuples out. No global state dependencies.

Contracts:

- require: log_probs.ndim == 2
- require: beam_width > 0
- ensure: len(result) <= beam_width

Witness: passing a perfectly clean, unambiguous sequence recovers the exact sequence, while supplying ambiguous frames gracefully merges probabilities mathematically over multiple valid paths within the active beam constraints.

Dependencies: numpy, math

CDG stages covered: bengali_speech/ctc_decode, asl_fingerspelling/ctc_decode

Works cited

1. Audio Resampling: Part 2 | flyingSand - WordPress.com, accessed April 28, 2026, <https://christianfloisand.wordpress.com/2013/01/28/audio-resampling-part-2/>
2. resample — SciPy v1.17.0 Manual, accessed April 28, 2026, <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample.html>
3. resample_poly — SciPy v1.17.0 Manual, accessed April 28, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample_poly.html
4. PyTorch implementation of scipy.signal.resample and scipy.signal.resample_poly - GitHub Gist, accessed April 28, 2026, <https://gist.github.com/thomasbrandon/63d609f37f8e73c56f5a4c76260aeb28>
5. librosa.resample — librosa 0.10.2 documentation, accessed April 28, 2026, <https://librosa.org/doc/0.10.2/generated/librosa.resample.html>
6. numpy.lib.stride_tricks.sliding_window_view, accessed April 28, 2026,

- https://numpy.org/devdocs/reference/generated/numpy.lib.stride_tricks.sliding_window_view.html
7. Understanding Sliding Window in NumPy | by whyamit404 - Medium, accessed April 28, 2026,
<https://medium.com/@whyamit404/understanding-sliding-window-in-numpy-50c86cac822b>
 8. librosa.feature.melspectrogram — librosa 0.11.0 documentation, accessed April 28, 2026,
<https://librosa.org/doc/main/generated/librosa.feature.melspectrogram.html>
 9. Understanding the Mel Spectrogram | by Leland Roberts | Analytics Vidhya | Medium, accessed April 28, 2026,
<https://medium.com/analytics-vidhya/understanding-the-mel-spectrogram-fca2afa2ce53>
 10. HOW to get MFCC from an FFT on a signal? - Stack Overflow, accessed April 28, 2026,
<https://stackoverflow.com/questions/5835568/how-to-get-mfcc-from-an-fft-on-a-signal>
 11. How to Create & Understand Mel-Spectrograms | by Christopher Lewis | Medium, accessed April 28, 2026,
<https://importchris.medium.com/how-to-create-understand-mel-spectrograms-ff7634991056>
 12. What is really the Mel-filterbank? - Signal Processing Stack Exchange, accessed April 28, 2026,
<https://dsp.stackexchange.com/questions/75296/what-is-really-the-mel-filterbank>
 13. MelBands — Essentia 2.1-beta6-dev documentation, accessed April 28, 2026,
https://essentia.upf.edu/reference/streaming_MelBands.html
 14. Digital signal processing — numpy-ml 0.1.0 documentation, accessed April 28, 2026,
https://numpy-ml.readthedocs.io/en/latest/numpy_ml.preprocessing.dsp.html
 15. Mel scale - Wikipedia, accessed April 28, 2026,
https://en.wikipedia.org/wiki/Mel_scale
 16. 【Analytics】Mel Spectrogram explanation - DEV Community, accessed April 28, 2026,
<https://dev.to/moyuto/analytics-mel-spectrogram-explanation-4ip7>
 17. strange scaling of slaney mels - Google Groups, accessed April 28, 2026,
<https://groups.google.com/g/librosa/c/tB1ExoTg-ho>
 18. librosa.filters.mel — librosa 0.11.0 documentation, accessed April 28, 2026,
<https://librosa.org/doc/main/generated/librosa.filters.mel.html>
 19. Speech Processing for Machine Learning: Filter banks, Mel-Frequency Cepstral Coefficients (MFCCs) and What's In-Between | Haytham Fayek, accessed April 28, 2026,
<https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html>
 20. Getting to Know the Mel Spectrogram | by Dalya Gartzman | TDS Archive | Medium, accessed April 28, 2026,

<https://medium.com/data-science/getting-to-know-the-mel-spectrogram-31bca3e2d9d0>

21. melSpectrogram - Mel spectrogram - MATLAB - MathWorks, accessed April 28, 2026, <https://www.mathworks.com/help/audio/ref/melspectrogram.html>
22. numpy-ml/numpy_ml/preprocessing/dsp.py at master - GitHub, accessed April 28, 2026, https://github.com/ddbourgin/numpy-ml/blob/master/numpy_ml/preprocessing/dsp.py
23. MFCC implementation and tutorial - Kaggle, accessed April 28, 2026, <https://www.kaggle.com/code/ilyamich/mfcc-implementation-and-tutorial>
24. dct — SciPy v1.17.0 Manual, accessed April 28, 2026, <https://docs.scipy.org/doc/scipy/reference/generated/scipy.fft.dct.html>
25. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition, accessed April 28, 2026, <https://courses.grainger.illinois.edu/ECE590SIC/sp2024/SpecAugment.pdf>
26. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition, accessed April 28, 2026, <https://arxiv.org/abs/1904.08779>
27. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition - ISCA Archive, accessed April 28, 2026, https://www.isca-archive.org/interspeech_2019/park19e_interspeech.pdf
28. MichaelisTrofficus/spec_augment: Tensorflow Layer that implements the SpecAugment technique - GitHub, accessed April 28, 2026, https://github.com/MichaelisTrofficus/spec_augment
29. Brief Review — SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition | by Sik-Ho Tsang, accessed April 28, 2026, <https://sh-tsang.medium.com/brief-review-specaugment-a-simple-data-augmentation-method-for-automatic-speech-recognition-1ceddfe24e2d>
30. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition - GitHub, accessed April 28, 2026, <https://github.com/pyyush/SpecAugment>
31. Simple implementation of median filter in python to remove noise from the images. - GitHub, accessed April 28, 2026, <https://github.com/MeteHanC/Python-Median-Filter>
32. Median Filter Tutorial for Noisy Signals - YouTube, accessed April 28, 2026, <https://www.youtube.com/watch?v=JB3ivVORYw0>
33. Signal processing (scipy.signal) — SciPy v1.17.0 Manual, accessed April 28, 2026, <https://docs.scipy.org/doc/scipy/reference/signal.html>
34. Loudness Normalization in Accordance with EBU R 128 Standard - MATLAB & Simulink, accessed April 28, 2026, <https://www.mathworks.com/help/audio/ug/loudness-normalization-in-accordance-with-ebu-r-128-standard.html>
35. pyloudnorm - PyPI, accessed April 28, 2026, <https://pypi.org/project/pyloudnorm/>
36. audiomentations 0.24.0 - PyPI, accessed April 28, 2026, <https://pypi.org/project/audiomentations/0.24.0/>
37. EBU R 128 - Wikipedia, accessed April 28, 2026,

- https://en.wikipedia.org/wiki/EBU_R_128
38. weightingFilter - Frequency-weighted filter - MATLAB - MathWorks, accessed April 28, 2026,
<https://www.mathworks.com/help/audio/ref/weightingfilter-system-object.html>
 39. MiRA - FLUX:: Documentation, accessed April 28, 2026,
<https://doc.flux.audio/mira/MiRA.pdf>
 40. LoudnessEBUR128 — Essentia 2.1-beta6-dev documentation, accessed April 28, 2026, https://essentia.upf.edu/reference/streaming_LoudnessEBUR128.html
 41. integratedLoudness - Measure integrated loudness and loudness range - MATLAB - MathWorks, accessed April 28, 2026,
<https://www.mathworks.com/help/audio/ref/integratedloudness.html>
 42. pyloudnorm/pyloudnorm/meter.py at master · csteinmetz1/pyloudnorm - GitHub, accessed April 28, 2026,
<https://github.com/csteinmetz1/pyloudnorm/blob/master/pyloudnorm/meter.py>
 43. Audio-Masking-Methods/MaskingMethods.py at master - GitHub, accessed April 28, 2026,
<https://github.com/ashishpatel26/Audio-Masking-Methods/blob/master/MaskingMethods.py>
 44. Consistent Wiener Filtering for Audio Source Separation - Mitsubishi Electric Research Laboratories, accessed April 28, 2026,
<https://www.merl.com/publications/docs/TR2012-090.pdf>
 45. sigsep/norbert: Painless Wiener filters for audio separation - GitHub, accessed April 28, 2026, <https://github.com/sigsep/norbert>
 46. Separate vocals from a track using python - DEV Community, accessed April 28, 2026,
<https://dev.to/highcenburg/separate-vocals-from-a-track-using-python-4lb5>
 47. On the Use of Masking Filters in Sound Source Separation - Arrow@TU Dublin, accessed April 28, 2026,
<https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1087&context=argcon>
 48. A Rule-Based Grapheme-to-Phoneme Conversion System - MDPI, accessed April 28, 2026, <https://www.mdpi.com/2076-3417/12/5/2758>
 49. (PDF) Gi2Pi Rule-based, index-preserving grapheme-to-phoneme transformations Rule-based, index-preserving grapheme-to-phoneme transformations - ResearchGate, accessed April 28, 2026,
https://www.researchgate.net/publication/361064040_Gi2Pi_Rule-based_index-preserving_grapheme-to-phoneme_transformations_Rule-based_index-preserving_grapheme-to-phoneme_transformations
 50. Gi2Pi: rule-based, index-preserving grapheme-to-phoneme transformations, accessed April 28, 2026,
<https://nrc-publications.canada.ca/eng/view/object/?id=de4b961d-54bf-4187-a3fc-d875ac285e79>
 51. Dynamic time warping - Wikipedia, accessed April 28, 2026,
https://en.wikipedia.org/wiki/Dynamic_time_warping
 52. Dynamic Time Warping (DTW) - AudioLabs Erlangen, accessed April 28, 2026,
https://www.audiolabs-erlangen.de/resources/MIR/FMP/C3/C3S2_DTWbasic.html

53. The dtw-python package 1.7.4 documentation, accessed April 28, 2026, <https://dynamictimewarping.github.io/py-api/html/api/dtw.dtw.html>
54. Treora/cdtw: NumPy-based constrained Dynamic Time Warping (cDTW) module - GitHub, accessed April 28, 2026, <https://github.com/Treora/cdtw>
55. An introduction to Dynamic Time Warping - Romain Tavenard, accessed April 28, 2026, <https://rtavenar.github.io/blog/dtw.html>
56. Super Monotonic Alignment Search - arXiv, accessed April 28, 2026, <https://arxiv.org/html/2409.07704v1>
57. Monotonic Alignment Search (MAS) - learnius, accessed April 28, 2026, [https://learnius.com/slp/9+Speech+Synthesis/2+Advanced+Topics/3+End-to-End+Models/Monotonic+Alignment+Search+\(MAS\)](https://learnius.com/slp/9+Speech+Synthesis/2+Advanced+Topics/3+End-to-End+Models/Monotonic+Alignment+Search+(MAS))
58. Glow-TTS: A Generative Flow for Text-to-Speech via Monotonic Alignment Search, accessed April 28, 2026, <https://proceedings.neurips.cc/paper/2020/file/5c3b99e8f92532e5ad1556e53cea00c-Paper.pdf>
59. glowTTS | gradTTS | VITS (Monotonic Alignment Search MAS) · coqui-ai TTS · Discussion #2025 - GitHub, accessed April 28, 2026, <https://github.com/coqui-ai/TTS/discussions/2025>
60. Glow-TTS: A Generative Flow for Text-to-Speech via Monotonic Alignment Search, accessed April 28, 2026, <https://proceedings.neurips.cc/paper/2020/hash/5c3b99e8f92532e5ad1556e53cea00c-Abstract.html>
61. monotonic-alignment-search - PyPI, accessed April 28, 2026, <https://pypi.org/project/monotonic-alignment-search/>
62. Sequence Modeling with CTC - Distill.pub, accessed April 28, 2026, <https://distill.pub/2017/ctc/>
63. speechbrain.decoders.ctc module - Read the Docs, accessed April 28, 2026, <https://speechbrain.readthedocs.io/en/latest/API/speechbrain.decoders.ctc.html>
64. CTC Networks and Language Models: Prefix Beam Search Explained - Medium, accessed April 28, 2026, <https://medium.com/corti-ai/ctc-networks-and-language-models-prefix-beam-search-explained-c11d1ee23306>
65. Can prefix beam search commonly used in speech recognition with CTC be implemented in such a simpler way? - Stack Overflow, accessed April 28, 2026, <https://stackoverflow.com/questions/60099279/can-prefix-beam-search-commonly-used-in-speech-recognition-with-ctc-be-implement>
66. E2E-ASR/ctc_decoder.py at master - GitHub, accessed April 28, 2026, https://github.com/HawkAaron/E2E-ASR/blob/master/ctc_decoder.py
67. Example CTC Decoder in Python - GitHub Gist, accessed April 28, 2026, https://gist.github.com/awni/56369a90d03953e370f3964c826ed4b0?permalink_comment_id=2322162
68. CTC Prefix Beam Search Decoding Algorithm with Language Model, accessed April 28, 2026, <https://zhaozeyu1995.github.io/CTC-Prefix-Beam-Search-Decoding-Algorithm-with-Language-Model/>